

 Universidad de los Andes Facultad de Ingeniería	Ingeniería de Sistemas y Computación ISIS-1611: Inteligencia artificial Semestre: 2026-19 Créditos: 3	
---	--	---

Taller 1: Logística Panini para la Copa Mundial 2026

La Copa Mundial de Fútbol 2026 se acerca y Panini debe distribuir sobres de láminas del álbum oficial a fans distribuidos por toda Colombia. La operación inicia en una bodega principal y debe decidir rutas eficientes sobre la red vial nacional para cumplir con la demanda en tiempo y con el menor costo posible.

La misión es crítica: cada entrega retrasada reduce la satisfacción de los aficionados, cada kilómetro innecesario incrementa los costos logísticos y cada decisión de ruta ineficiente compromete la cobertura nacional del lanzamiento. La flota debe navegar la red vial modelada para llegar a uno o varios destinos de entrega.

Cada decisión de ruta importa. Los camiones deben minimizar kilómetros, número de tramos o el costo total de un tour con varias entregas, según el problema modelado. Elegir un algoritmo inadecuado o una heurística poco informativa puede expandir decenas de miles de nodos incluso en rutas aparentemente simples.

Su responsabilidad es diseñar e implementar la lógica de búsqueda que permita a Panini encontrar rutas de entrega de manera óptima, comparando diferentes estrategias de búsqueda, entendiendo cuándo cada una es más eficiente y desarrollando heurísticas sofisticadas que permitan planificar entregas simultáneas en múltiples destinos.

Objetivos de aprendizaje

Al completar este taller, ustedes serán capaces de:

1. Modelar problemas de búsqueda como agentes que exploran estados y caminos posibles dentro de un grafo definido.
2. Implementar algoritmos clásicos de búsqueda desinformada (DFS, BFS, UCS, iterative deepening) e informada (A^*).
3. Diseñar heurísticas admisibles y consistentes aplicadas a escenarios reales de toma de decisiones logísticas.
4. Analizar el trade-off entre optimalidad, costo computacional y tiempo de respuesta, comparando diferentes enfoques de búsqueda.
5. Desarrollar pensamiento crítico y estructurado al traducir un contexto real de distribución nacional a un modelo computacional.

Escenario del Problema

La red vial de Colombia se modela como un grafo ponderado no dirigido extraído de OpenStreetMap. La flota de Panini cuenta con las siguientes características operacionales:

- Puede desplazarse únicamente por aristas existentes en el grafo vial entregado.
- No puede atravesar nodos aislados ni salirse de la red vial modelada.
- Puede realizar múltiples entregas en una misma ruta, pero debe llegar hasta cada destino pendiente.
- Debe minimizar un criterio de costo que depende del tipo de problema modelado (kilómetros o número de tramos).

Problema	Criterio de costo	Descripción
SingleDeliveryProblem	Kilómetros	Una bodega y un destino. Minimiza la distancia total recorrida.
FewestStopsDeliveryProblem	Número de tramos	Una bodega y un destino. Cada arista tiene costo unitario.
MultiDeliveryProblem	Kilómetros	Varias entregas en cualquier orden. Variante de TSP sobre el grafo vial.

El grafo entregado tiene 41.718 nodos y 64.126 aristas no dirigidas, una sola componente conexa, costos simétricos en kilómetros y diámetro aproximado de 295 tramos.

Estructura del Proyecto

Ustedes reciben un esqueleto base funcional que incluye toda la infraestructura de carga del grafo, definición de problemas y visualización. La responsabilidad del grupo es completar los algoritmos de búsqueda y las heurísticas específicas en dos archivos designados:

- `algorithms/search.py`: Contiene las implementaciones de DFS, BFS, UCS, A* e iterative deepening. Aquí escribirán las funciones principales que resuelven los problemas de navegación logística.
- `algorithms/heuristics.py`: Define las funciones heurísticas (como `straightLineHeuristic` y `multiDeliveryHeuristic`) para búsqueda informada. Aquí adaptan los algoritmos genéricos al contexto de entregas sobre el grafo vial.

Para facilitar la implementación, ustedes cuentan con el archivo `algorithms/utils.py` que proporciona estructuras de datos eficientes como Stack, Queue y PriorityQueue. Aunque no deben modificar los demás archivos, pueden consultarlos para entender detalles de implementación:

- Módulo `algorithms/`:
 - `problems.py`: Define la estructura de un problema de búsqueda y especifica `SingleDeliveryProblem`, `FewestStopsDeliveryProblem` y `MultiDeliveryProblem`.
 - `utils.py`: Implementa Stack, Queue, PriorityQueue y utilidades de apoyo.
- Módulo `graph/`:
 - `road_graph.py`: Carga y valida el grafo vial desde `data/colombia_roads.json`.
- Módulo `visualization/`:
 - `map_view.py`: Genera un mapa interactivo con tiles de OpenStreetMap. Se activa con `--show`.
- Carpeta `data/`:
 - `colombia_roads.json` (grafo real) e `instances.json` (instancias de ejemplo).
- Carpeta `notebooks/`:
 - `graph_visualization.ipynb` para explorar el grafo y consultar IDs de nodos.
- Archivo `main.py`: Interfaz de línea de comandos e impresión de métricas de desempeño.

Las secciones por completar están delimitadas por la etiqueta `### YOUR CODE HERE ###`. Al ejecutar `main.py`, el resumen reporta costo, acciones, nodos expandidos, frontera máxima y tiempo. Los nodos expandidos se cuentan en `getSuccessors`; la frontera máxima requiere llamar `_remember_frontier` en DFS, BFS, UCS y A*.

Ejecución

El programa se ejecuta con el siguiente formato general:

```
python main.py [OPCIONES]
```

Existen dos modos equivalentes para configurar una corrida: (1) argumentos en la línea de comandos y (2) instancias nombradas en `data/instances.json`. Las instancias incluidas son ejemplos de referencia; se espera que ustedes diseñen instancias propias tras inspeccionar el grafo en el notebook, creando casos que permitan contrastar el comportamiento teórico esperado de cada algoritmo.

Opción	Descripción	Default	Ejemplo
<code>--graph</code>	Ruta al archivo JSON del grafo vial.	<code>data/colombia_roads.json</code>	<code>--graph data/colombia_roads.json</code>
<code>--problem</code>	Tipo de problema: single, stops o multi.	single	<code>--problem stops</code>
<code>--algorithm</code>	Función de búsqueda: dfs, bfs, ucs, astar, ids.	ucs	<code>-a astar</code>
<code>--heuristic</code>	Heurística para A*.	Automática	<code>--heuristic straightLineHeuristic</code>
<code>--start</code>	ID del nodo de inicio.	n1037511	<code>--start n1037511</code>
<code>--goal</code>	ID del destino para single y stops.	n39739	<code>--goal n15395</code>
<code>--deliveries</code>	IDs de entrega separados por comas (multi).	Ninguno	<code>--deliveries n39739,n15395</code>

--instance	Nombre de un caso en data/instances.json.	Ninguno	--instance multi_andes_caribe
--max-depth	Tope opcional de profundidad para ids.	Sin tope	--max-depth 6
--show	Abre mapa interactivo con ruta y nodos expandidos.	Desactivado	--show
--help	Muestra la ayuda con todas las opciones disponibles.	—	--help

Tip: Analicen el código de algorithms/search.py y las definiciones en algorithms/problems.py. Sus algoritmos de búsqueda deben devolver una secuencia de acciones, donde cada acción es el ID del nodo siguiente en la ruta. Abran notebooks/graph_visualization.ipynb para elegir nodos antes de definir sus propias instancias.

Nota: La visualización con --show abre un archivo HTML temporal en el navegador usando Folium/Leaflet. Si su entorno no permite abrir ventanas automáticamente, el programa imprimirá en consola la ruta del archivo para que puedan abrirlo manualmente. Se recomienda usar --show para verificar visualmente las rutas encontradas.

Punto 1: Exploración Inicial de la Red Vial (20%)

Panini recibe una solicitud urgente de entrega a un fan en un nodo específico de la red. Primero se desea encontrar cualquier ruta válida rápidamente; después, una ruta con el menor número de tramos posibles. Para minimizar el número de tramos, modelen el problema como FewestStopsDeliveryProblem.

- Implementen el algoritmo depthFirstSearch.
- Implementen el algoritmo breadthFirstSearch.
- Prueben los algoritmos sobre varios pares origen-destino. ¿Qué diferencias encuentra entre los resultados de ambos algoritmos?

Punto 2: Ruta de Distancia Mínima (15%)

El menor número de tramos no siempre implica la menor distancia recorrida. Ahora Panini debe minimizar los kilómetros totales de la ruta de entrega. Para esta misión deben modelar el problema como SingleDeliveryProblem.

- Implementen el algoritmo uniformCostSearch.
- Prueben su algoritmo sobre varias instancias propias de una sola entrega. ¿Cómo se diferencian las rutas generadas por UCS con respecto a los algoritmos de búsqueda desinformada?

Punto 3: Búsqueda Informada Eficiente (20%)

Panini dispone de las coordenadas geográficas de cada nodo y puede estimar la distancia en línea recta hacia el destino. Pueden usar esta información para guiar la búsqueda y encontrar soluciones óptimas de manera mucho más rápida.

- a) Implementen la heurística `straightLineHeuristic` y el algoritmo `aStarSearch`.
- b) Prueben su algoritmo sobre rutas largas de una sola entrega con `SingleDeliveryProblem`. Verifiquen en varios casos de interés que A* retorna el mismo costo que UCS.

Punto 4: Búsqueda con Profundidad Iterativa (15%)

En algunas situaciones se desea el comportamiento exploratorio de DFS, pero con garantías de encontrar soluciones de profundidad mínima cuando la profundidad óptima es desconocida.

- a) Implementen `depthLimitedSearch` e `iterativeDeepeningSearch`.
- b) Prueben IDS sobre instancias de `FewestStopsDeliveryProblem` y compárenlo con BFS y DFS.
- c) Analicen el costo en tiempo de repetir búsquedas con límites de profundidad crecientes.

Punto 5: Entregas Múltiples (20%)

Se han recibido pedidos de múltiples fans dispersos por distintas regiones del país. La flota debe visitarlos todos en algún orden para minimizar el costo total.

- a) Revisen la definición de un `MultiDeliveryProblem`, diseñen una heurística admisible en `multiDeliveryHeuristic` y pruébenla con el algoritmo A*.
- b) Evalúen sus heurísticas en instancias de varias entregas. Pueden comenzar con las instancias suministradas de `multi_andes_caribe` y `multi_national_challenge`. Comparen `multiDeliveryHeuristic` contra `nullHeuristic` en costo, tiempo y nodos expandidos.
- c) Experimenten computacionalmente aumentando la cantidad y la complejidad de las entregas.

Punto 6: Reflexión sobre el trabajo realizado (10%)

Panini desea mejorar su sistema de planificación logística para incrementar la eficiencia de futuras campañas de distribución. Con este fin, les ha solicitado un análisis a partir de la experiencia del grupo diseñando algoritmos de búsqueda. Contesten las siguientes preguntas en un documento PDF:

- a) Para cada uno de los puntos anteriores indiquen claramente en el contexto del problema:
 - Complejidad en espacio (Notación O).
 - Complejidad en tiempo (Notación O).
 - ¿El algoritmo es completo?
 - ¿El algoritmo encuentra la solución óptima siempre o en cuáles condiciones?
 - Para los puntos 3 y 5 indiquen si las heurísticas son consistentes.

Se espera que en su análisis comparen cualitativamente el desempeño de los distintos algoritmos en las instancias que diseñen, así como que comparen cuantitativamente métricas como el costo de la solución, la cantidad de nodos expandidos, el tamaño máximo de la frontera y el tiempo de ejecución reportado por el programa.

- b) Escriban una reflexión concisa sobre la co-construcción de la solución final con apoyo de la IA (ver política de uso de la IA más adelante), indicando, por ejemplo, qué aprendizajes obtuvieron al ver las correcciones hechas por la IA, si éstas fueron básicas o solo mejoras secundarias; la facilidad o dificultad para crear el prompt adecuado, si la solución propuesta fue fácil de entender (¿la podrían explicar en una sustentación?). Si no hicieron ninguna iteración con IA, indiquen por qué no les pareció necesario ni útil. Aquí no hay respuestas correctas ni incorrectas. Este punto busca que ustedes se tomen el tiempo de reflexionar sobre el uso de la IA en tareas de programación.

Política de Uso de IA Generativa y presentación en el trabajo a entregar

Para este taller se espera que ustedes desarrollen una primera versión de la solución de forma completamente autónoma, usando su propio criterio, conocimiento y creatividad antes de recurrir a herramientas de IA. La IA puede emplearse después para mejoras puntuales, refactorización, comentarios de calidad o apoyo en la corrección de errores, pero nunca como sustituto del esfuerzo personal ni como generador principal del código. Usen estas herramientas con criterio profesional, asumiendo responsabilidad sobre su proceso de aprendizaje y entendiendo que lo que más valor tiene aquí es el camino que ustedes recorren para llegar a la solución, no solo el resultado final.

Todos los prompts y las versiones de código generadas con apoyo de IA deben quedar registrados dentro de los archivos del proyecto. Si utilizan IA para refactorizar u optimizar su solución, incluyan en el archivo: (1) la versión inicial del código, (2) los prompts que utilizaron para refinarla y (3) la versión final del código. Al finalizar, dejen como código activo únicamente la versión final y conserven la versión inicial y todos los prompts en forma de comentarios dentro del mismo archivo.

Entrega del trabajo

- Guarden todo su trabajo (incluyendo el documento PDF) en una carpeta comprimida (.zip)
- Suban el trabajo al espacio correspondiente en Bloque Neón a más tardar la fecha y hora indicadas (un solo miembro del grupo debe subir el trabajo).
- Recuerden incluir los nombres y códigos de todos los integrantes del grupo.

Nota: Al enviar su solución, ustedes declaran que el código entregado en la primera versión de cada punto es de autoría del grupo y que las versiones que utilizan IA fueron la respuesta a los prompts incluidos, que comprenden el funcionamiento en todas las versiones y que aceptan que este material pueda ser revisado, ejecutado y evaluado por el equipo docente para efectos académicos y de verificación de originalidad.